D Y PATIL
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

INFORMATION SECURITY LABORATORY MANUAL

Lab Manual

Third Year Semester-VI

Even Semester

Institutional Vision, Mission and Quality Policy

Our Vision

To foster and permeate higher and quality education with value added engineering, technology programs, providing all facilities in terms of technology and platforms for all round development with societal awareness and nurture the youth with international competencies and exemplary level of employability even under highly competitive environment so that they are innovative adaptable and capable of handling problems faced by our country and world at large.

RAIT's firm belief in new form of engineering education that lays equal stress on academics and leadership building extracurricular skills has been a major contribution to the success of RAIT as one of the most reputed institution of higher learning. The challenges faced by our country and world in the 21 Century needs a whole new range of thought and action leaders, which a conventional educational system in engineering disciplines are ill equipped to produce. Our reputation in providing good engineering education with additional life skills ensure that high grade and highly motivated students join us. Our laboratories and practical sessions reflect the latest that is being followed in the Industry. The project works and summer projects make our students adept at handling the real life problems and be Industry ready. Our students are well placed in the Industry and their performance makes reputed companies visit us with renewed demands and vigour.

Our Mission

The Institution is committed to mobilize the resources and equip itself with men and materials of excellence thereby ensuring that the Institution becomes pivotal center of service to Industry, academia, and society with the latest technology. RAIT engages different platforms such as technology enhancing Student Technical Societies, Cultural platforms, Sports excellence centers, Entrepreneurial Development Center and Societal Interaction Cell. To develop the college to become an autonomous Institution & deemed university at the earliest with facilities for advanced research and development

programs on par with international standards. To invite international and reputed national Institutions and Universities to collaborate with our institution on the issues of common interest of teaching and learning sophistication.

RAIT's Mission is to produce engineering and technology professionals who are innovative and inspiring thought leaders, adept at solving problems faced by our nation and world by providing quality education.

The Institute is working closely with all stake holders like industry, academia to foster knowledge generation, acquisition, dissemination using best available resources to address the great challenges being faced by our country and World. RAIT is fully dedicated to provide its students skills that make them leaders and solution providers and are Industry ready when they graduate from the Institution.

We at RAIT assure our main stakeholders of students 100% quality for the programmes we deliver. This quality assurance stems from the teaching and learning processes we have at work at our campus and the teachers who are handpicked from reputed institutions IIT/NIT/MU, etc. and they inspire the students to be innovative in thinking and practical in approach. We have installed internal procedures to better skills set of instructors by sending them to training courses, workshops, seminars and conferences. We have also a full fledged course curriculum and deliveries planned in advance for a structured semester long programme. We have well developed feedback system employers, alumni, students and parents from to fine tune Learning and Teaching processes. These tools help us to ensure same quality of teaching independent of any individual instructor. Each classroom is equipped with Internet and other digital learning resources.

The effective learning process in the campus comprises a clean and stimulating classroom environment and availability of lecture notes and digital resources prepared by instructor from the comfort of home. In addition student is provided with good number of assignments that would trigger his thinking process. The testing process involves an objective test paper that would gauge the understanding of concepts by the students. The quality assurance process also ensures that the learning process is effective. The summer internships and project work based training ensure learning process to include practical and industry relevant aspects. Various technical events, seminars and conferences make the student learning complete.

Our Quality Policy

ज्ञानधीनं जगत् सर्वम।

Knowledge is supreme.

Our Quality Policy

It is our earnest endeavour to produce high quality engineering professionals who are innovative and inspiring, thought and action leaders, competent to solve problems faced by society, nation and world at large by striving towards very high standards in learning, teaching and training methodologies.

Our Motto: If it is not of quality, it is NOT RAIT!

Departmental Vision, Mission

Vision

To impart higher and quality education in computer science with value added engineering and technology programs to prepare technically sound, ethically strong engineers with social awareness. To extend the facilities, to meet the fast changing requirements and nurture the youths with international competencies and exemplary level of employability and research under highly competitive environments.

Mission

To mobilize the resources and equip the institution with men and materials of excellence to provide knowledge and develop technologies in the thrust areas of computer science and Engineering. To provide the diverse platforms of sports, technical, cocurricular and extracurricular activities for the overall development of student with ethical attitude. To prepare the students to sustain the impact of computer education for social needs encompassing industry, educational institutions and public service. To collaborate with IITs, reputed universities and industries for the technical and overall upliftment of students for continuing learning and entrepreneurship.

Departmental Program Educational Objectives (PEOs)

---

1. **Learn and Integrate**

   To provide Computer Engineering students with a strong foundation in the mathematical, scientific and engineering fundamentals necessary to formulate, solve and analyze engineering problems and to prepare them for graduate studies.

2. **Think and Create**

   To develop an ability to analyze the requirements of the software and hardware, understand the technical specifications, create a model, design, implement and verify a computing system to meet specified requirements while considering real-world constraints to solve real world problems.

3. **Broad Base**

   To provide broad education necessary to understand the science of computer engineering and the impact of it in a global and social context.

4. **Techno-leader**

   To provide exposure to emerging cutting edge technologies, adequate training & opportunities to work as teams on multidisciplinary projects with effective communication skills and leadership qualities.

5. **Practice citizenship**

   To provide knowledge of professional and ethical responsibility and to contribute to society through active engagement with professional societies, schools, civic organizations or other community activities.

6. **Clarify Purpose and Perspective**

   To provide strong in-depth education through electives and to promote student awareness on the life-long learning to adapt to innovation and change, and to be successful in their professional work or graduate studies.

Departmental Program Outcomes (POs)

PO1. Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2. Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3. Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4. Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5. Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6. The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7. Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8. Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9. Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10. Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to

comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11. Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12. Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Program Specific Outcomes:

PSO1: To build competencies towards problem solving with an ability to understand, identify, analyze and design the problem, implement and validate the solution including both hardware and software.

PSO2: To build appreciation and knowledge acquiring of current computer techniques with an ability to use skills and tools necessary for computing practice.

PSO3: To be able to match the industry requirements in the area of computer science and engineering. To equip skills to adopt and imbibe new technologies.

Index

Sr. No.	Contents	Page No.
1.	Experiment No. 1	
2.	Experiment No. 2	
3.	Experiment No. 3	
4.	Experiment No. 4	
5.	Experiment No. 5	
6.	Experiment No. 6	
7.	Experiment No. 7	
8.	Experiment No. 8	
9.	Experiment No. 9	
10.	Experiment No. 10	
11	Case Study	

List of Experiments

Sr. No.	Experiments Name	Course Outcome
1	Security Parameter Analysis	CO1
2	Implementing Access Control Models	CO2
3	Security Policy Simulation	CO3
4	Secure System Design	CO4
5	Vulnerability and Intrusion Detection	CO5
6	Data Privacy and Encryption	CO3
7	Digital Forensics Basics	CO6
8	Network and Program Security	CO5
9	Operating Systems Security	CO4
10	Database Security and Auditing	CO6
11	Case Study: Security Breach Analysis	

Experiment Plan, Course Objectives & Course Outcome

Course Objectives:

1.	To comprehend the core concepts of confidentiality, integrity, and availability (CIA), as well as security violations, threats, and life cycle processes.
2.	To analyze and implement access control models, including discretionary, mandatory, role-based, and advanced models, for secure information flow and identity representation.
3.	To design and enforce security policies that ensure confidentiality, integrity, and compliance with international standards, while incorporating assurance methods for robust system evaluation.
4.	To address advanced topics such as malicious logic, intrusion detection, data privacy, digital forensics, and enterprise security in real-world applications.

Course Outcomes:

CO1	Demonstrate an understanding of fundamental security parameters and life cycle processes for designing secure systems.
CO2	Analyze, design, and implement various access control models and temporal/spatio-temporal frameworks to secure data and resources.
CO3	Design confidentiality and integrity policies, analyze hybrid policies, and ensure compliance with international security standards.
CO4	Use formal methods and assurance techniques to evaluate and ensure system security.
CO5	Identify vulnerabilities, detect intrusions, and audit systems to protect against malicious logic and emerging security threats.
CO6	Implement solutions to secure networks, operating systems, databases, and programs, with a focus on enterprise and forensics applications.

Module No.	Week No.	Experiments Name	Course Outcome
1	W1	Security Parameter Analysis	CO1
2	W2	Implementing Access Control Models	CO2
3	W3	Security Policy Simulation	CO3
4	W4	Secure System Design	CO4
5	W5	Vulnerability and Intrusion Detection	CO5
6	W6	Data Privacy and Encryption	CO3
7	W7	Digital Forensics Basics	CO6
8	W8	Network and Program Security	CO5
9	W9	Operating Systems Security	CO4
10	W10	Database Security and Auditing	CO6
11	W11	Case Study: Security Breach Analysis	

Mapping of Course outcomes with Program outcomes

Subject Weight		Contribution to Program outcomes (PO)											
		1	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**
THEORY 20%	Demonstrate an understanding of fundamental security parameters and life cycle processes for designing secure systems.	1	1	1	1		1	1		1		1	1
	Analyze, design, and implement various access control models and temporal/spatio-temporal frameworks to secure data and resources.	2	2	1	2							2	2
	Design confidentiality and integrity policies, analyze hybrid policies, and ensure compliance with international security	1	2	1			1	1				2	1
	Use formal methods and assurance techniques to evaluate and ensure system security.	2	1	2	2							1	2
	Identify vulnerabilities, detect intrusions, and audit systems to protect against malicious logic and emerging security threats.	1	1	1	2	1	1	1				1	1
	Implement solutions to secure networks, operating systems, databases, and programs, with a focus on enterprise and forensics applications.	2	1	1	2	1					1	1	2
		1	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**
PRATICAL / ORAL + TERM WORK 80%	Demonstrate an understanding of fundamental security parameters and life cycle processes for designing secure systems.	1	1	1	1		1	1		1		1	1
	Analyze, design, and implement various access control models and temporal/spatio-temporal frameworks to secure data and resources.	2	2	1	2							2	2
	Design confidentiality and integrity policies, analyze hybrid policies, and ensure compliance with international security	1	2	1			1	1				2	1
	Use formal methods and assurance techniques to evaluate and ensure system security.	2	1	2	2							1	2
	Identify vulnerabilities, detect intrusions, and audit systems to protect against malicious logic and emerging security threats.	1	1	1	2	1	1	1				1	1
	Implement solutions to secure networks, operating systems, databases, and programs, with a focus on enterprise and forensics applications.	2	1	1	2	1					1	1	2

Mapping of Course outcomes with Program Specific outcomes:

Course Outcomes		Contribution to Program Specific outcomes (PSO)		
		1	2	3
CO1	Demonstrate an understanding of fundamental security parameters and life cycle processes for designing secure systems.	2	1	
CO2	Analyze, design, and implement various access control models and temporal/spatio-temporal frameworks to secure data and resources.	3		
CO3	Design confidentiality and integrity policies, analyze hybrid policies, and ensure compliance with international security standards.	1	1	1
CO4	Use formal methods and assurance techniques to evaluate and ensure system security.	2	1	
CO5	Identify vulnerabilities, detect intrusions, and audit systems to protect against malicious logic and emerging security threats.	2	1	
CO6	Implement solutions to secure networks, operating systems, databases, and programs, with a focus on enterprise and forensics applications.	2		1

Mapping Course Outcomes (CO) - Program Outcomes (PO)

Study and Evaluation Scheme

Course Code	Course Name	Teaching Scheme			Credits Assigned			
CSL602	INFORMATION SECURITY LAB	Theory	Practical	Tutorial	Theory	Practical	Tutorial	Total
		--	02	--	--	01	--	01

Course Code	Course Name	Examination Scheme		
CSL602	INFORMATION SECURITY LAB	Term Work	Oral	Total
		25	25	50

Term Work:

Laboratory work will be based on above syllabus with minimum 10 experiments to be incorporated.

Laboratory work (experiments): …… (15) Marks.

Assignments: ……..... (05) Marks.

Attendance (Theory + Practical) ……(05) Marks

TOTAL: …… (25) Marks.

Practical / Oral Exam:

Practical / Oral Exam will be based on the CSC602 and CSL602 syllabus.

Information Security Lab

Experiment No.: 1

Security Parameter Analysis

Experiment No. 1

1. **Aim:** To explore the CIA triad (Confidentiality, Integrity, and Availability) and identify potential security violations through practical examples and system analysis.

2. **Objectives:**

- To understand the core principles of the CIA triad.
- To analyze scenarios of security violations affecting confidentiality, integrity, and availability.
- To gain hands-on experience in identifying and addressing security breaches.

3 Outcomes:

From this experiment, the student will be able to

- Understand the importance of the CIA triad in securing information systems.
- To identify security violations and classify them based on confidentiality, integrity, and availability.
- Develop basic skills for mitigating security violations using system tools.

4. **Hardware/Software Required :**

- A computer system with minimum specifications:
- Processor: 1 GHz or faster
- RAM: 4 GB or more
- Disk Space: 20 GB free
- Linux-based OS (e.g., Ubuntu, Fedora) or Windows OS
- Terminal/Command Prompt
- Wireshark (for network traffic analysis)
- Text editor (e.g., Vim, Nano, Notepad++)
- Access to system logs (/var/log/ in Linux or Event Viewer in Windows)

5. **Theory:**

The CIA triad forms the foundation of information security:

1. Confidentiality: Ensures that information is accessible only to authorized users. Security violations include data breaches, unauthorized access, and eavesdropping.

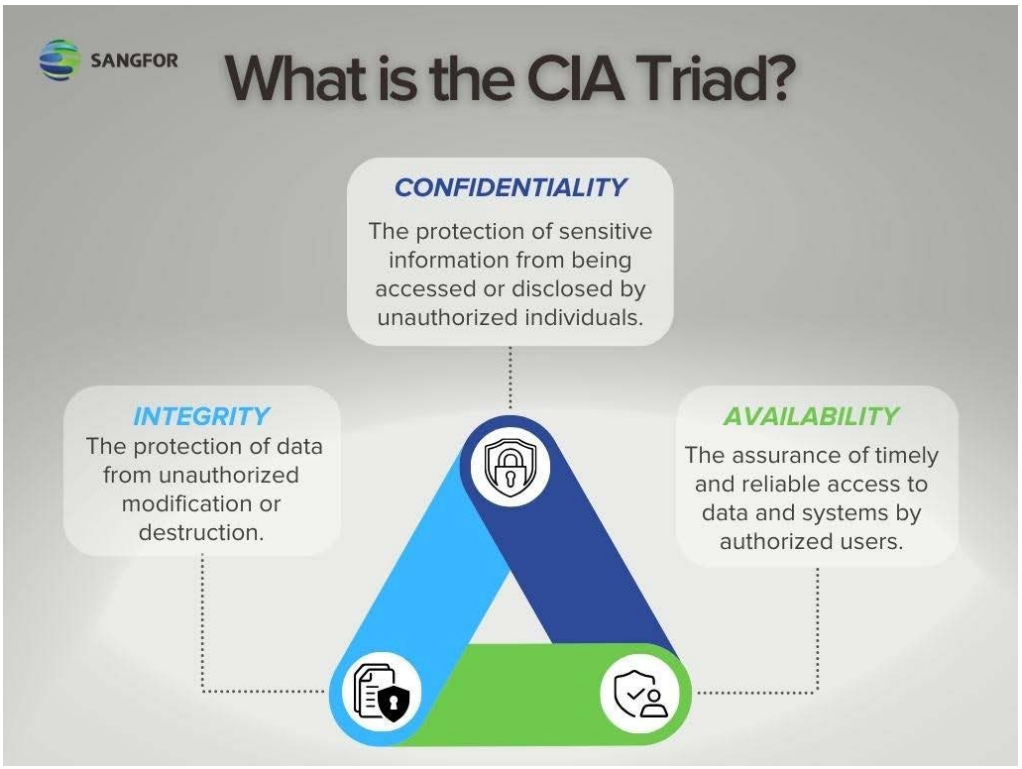
Example: A leaked password file compromises user privacy.

2. Integrity: Ensures that information is accurate and unaltered. Violations include unauthorized modifications, data corruption, or insertion of false data.

   Example: A malware attack modifying log files to hide unauthorized access.

3. Availability: Ensures that authorized users can access the information and resources when required. Violations include denial-of-service attacks or hardware failures.

Example: A DDoS attack making a website unavailable.



6. **Procedure/ Program:**

Lab Steps for Lab 1: Security Parameter Analysis

Step 1: Explore Confidentiality

Objective: Understand how confidentiality is maintained and identify security violations when unauthorized access occurs.

1. Create a File Containing Sensitive Information

   - Open a terminal or command prompt.
   - Create a text file named sensitive_file.txt:
     - Linux: echo "Confidential Data: Usernames and Passwords" > sensitive_file.txt
     - Windows: Open Notepad, type "Confidential Data: Usernames and Passwords", and save the file as sensitive_file.txt.

2. Restrict File Permissions

   - Set file permissions so only the owner can access it:
     - Linux: Run chmod 600 sensitive_file.txt.
       - Explanation: The 600 permission allows the owner to read/write the file but denies access to others.
     - Windows: Right-click the file → Properties → Security → Edit permissions → Deny access for all users except the owner.

3. Simulate Unauthorized Access

   - Switch to another user or simulate unauthorized access:
     - Linux: Use su or create a new user, then try accessing the file: cat sensitive_file.txt.
     - Windows: Switch user accounts or create a new user, then try opening the file.
   - Observe the error message (e.g., "Permission denied").
   - Discussion: Analyze how this protects confidentiality. Discuss real-world examples, such as protecting medical or financial records.

Step 2: Analyze Integrity

Objective: Understand how data integrity can be compromised and verify its integrity using hashing.

1. Create or Access a Log File

   - Use an existing log file or create a simulated one:
     - Linux: sudo nano /var/log/syslog (requires root access).
     - Windows: Open Event Viewer (eventvwr) or create a text file named

logfile.txt.

2. Modify the Log File (Simulate Unauthorized Changes)

   - Add or change log entries to simulate a security violation:
     - Linux: Edit the file: sudo nano /var/log/syslog → Add a fake entry: Jan 1 12:00:00 UnauthorizedAccess: Admin login.
     - Windows: Open logfile.txt in Notepad and add UnauthorizedAccess: Admin login.
3. Verify Integrity with Hashing

   - Calculate the file's hash before and after modification:
     - Linux: Use sha256sum logfile.txt and note the hash.
     - Windows: Use PowerShell: Get-FileHash .\logfile.txt -Algorithm SHA256

   - Observe the hash difference.
   - Discussion: Discuss how unauthorized changes compromise data integrity. Relate this to tampering scenarios in real-world applications, such as altering financial records or audit logs.

Step 3: Examine Availability
Objective: Explore how system availability is affected during an attack or resource overload.

1. Set Up a Simple Web Server (Optional)

   - Linux: Use Python to start a basic web server: python3 -m http.server 8080
   - Windows: Use IIS or WAMP/XAMPP to set up a local server.

2. Simulate Denial-of-Service (DoS) Attack

   - Use a tool to overload the server with requests:
     - Linux: Install and use ab (Apache Benchmark):
       ab -n 1000 -c 100 http://localhost:8080/
       -n: Total number of requests.
       -c: Number of concurrent requests.

     - Windows: Use a custom PowerShell script or any load-testing tool like JMeter.

3. Monitor Server Behavior

   - Observe the server response time during the attack:
     - Linux: Check server logs or terminal output for delays or errors.
     - Windows: Use Task Manager or Resource Monitor to track CPU and network usage.
   - Note any timeouts or connection refusals.

4. Restore Normal Operations

   - Stop the attack and ensure normal availability:
     - Linux: Terminate the ab command or server process (Ctrl+C).
     - Windows: Stop the server or restart it via IIS Manager.
   - Verify that the server is responsive again.

5. Discussion

   - Relate this to real-world examples of denial-of-service attacks on popular websites or applications. Discuss strategies to mitigate such attacks, such as rate-limiting, firewalls, or load balancers.

Step 4: Document Observations

a. For each step, document:
   - Actions performed.
   - Observations (e.g., error messages, response times).
   - Screenshots of commands and results.

b. Analyze how each violation impacts the overall security of the system.

7. Conclusion/Discussion:

The CIA triad is essential for securing information systems. Through this lab, students understood the importance of confidentiality, integrity, and availability in real-world scenarios. They identified potential security violations and learned basic techniques to mitigate these issues.

8. References:

- William Stallings, Cryptography and Network Security: Principles and Practice, Pearson Education.
- Matt Bishop, Computer Security: Art and Science, Addison-Wesley.
- Official Linux Documentation: https://www.kernel.org/doc/
- Wireshark User Guide: https://www.wireshark.org/docs/
- Microsoft Security Documentation: https://docs.microsoft.com/

Information Security Lab

Experiment No.: 2

Implementing Access Control Models (DAC, MAC, and RBAC)

Experiment No. 2

1. **Aim:** To configure and understand Discretionary Access Control (DAC), Mandatory Access Control (MAC), and Role-Based Access Control (RBAC) mechanisms in a Linux environment.

2. **Objectives:**

   - To understand the principles of DAC, MAC, and RBAC in access control.
   - To implement and configure these models in a controlled environment.
   - To analyze the advantages, limitations, and use cases of each access control model.

3. **Outcomes:**

   - Students will understand the theoretical foundation of DAC, MAC, and RBAC.
   - Students will configure and test these access control models using Linux tools and policies.
   - Students will evaluate and compare the practical applications of these models in real-world scenarios.

4. **Hardware/Software Required:**
   - **Hardware:**
     - A computer or virtual machine with minimum specifications:
     - Processor: 1 GHz or faster
     - RAM: 4 GB or more
     - Disk Space: 20 GB free
   - **Software:**
     - Linux-based OS (e.g., Ubuntu 20.04 or later, CentOS)
     - Terminal/Command Line Interface (CLI)
     - SELinux or AppArmor (for MAC)
     - chmod, setfacl, and chown commands (for DAC)
     - OpenLDAP or a Linux group-based system (for RBAC)

5. **Theory:**

Access Control Models:

1. Discretionary Access Control (DAC):

   - Definition: The owner of a resource decides who can access it and what permissions they have.
   - Example: File and directory permissions set by a user using chmod or chown.
   - Use Case: Suitable for environments with low-security requirements where flexibility is needed.

2. Mandatory Access Control (MAC):

    - Definition: Access is based on predefined security policies enforced by the system, regardless of resource ownership.
    - Example: SELinux or AppArmor policies.
    - Use Case: Suitable for high-security environments like government or military systems.

3. Role-Based Access Control (RBAC):

    - Definition: Access is assigned based on the user's role within an organization. Roles have predefined permissions.
    - Example: Admin roles having access to sensitive directories.
    - Use Case: Suitable for enterprise environments where role hierarchies exist.

6. Procedure/Program:

1. Discretionary Access Control (DAC)

    a. Create Files and Directories:
        - Command
          mkdir dac_demo
          cd dac_demo
          touch confidential.txt

        - What is happening?

            - mkdir dac_demo: Creates a directory named dac_demo.
            - touch confidential.txt: Creates an empty file named confidential.txt inside the directory.

    b. Set Permissions Using chmod:
        - Command:
          chmod 600 confidential.txt

        - The chmod 600 command changes the file's permissions:
            - Owner: Can read and write.
            - and Others: No permissions.

    c. Change File Ownership Using chown:
        - Commands:
          sudo adduser alice
          sudo chown alice:alice confidential.txt

        - What is happening?
            - sudo adduser alice: Creates a new user named Alice.
            - sudo chown alice:alice confidential.txt: Transfers ownership of the file confidential.txt to Alice.

d. Log in as Alice:
   - Command:
     su alice
     cat confidential.txt

   - What is happening?
     - su alice: Switches to the user Alice.
     - cat confidential.txt: Tries to read the file.

e. Results:
   - Alice, as the file owner, can access confidential.txt.
   - Other users (e.g., the default user) cannot read or write to the file because of the restrictive permissions (600).

2. Mandatory Access Control (MAC)

a. Enable SELinux:

   - Command : getenforce

   - What is happening?

     - Checks the status of SELinux (enforcing, permissive, or disabled).
   - If SELinux is not enabled:
     - sudo setenforce 1
     - This activates SELinux in "enforcing" mode.

b. Apply Security Context to a File:
   - Command: ls -Z confidential.txt
   - What is happening?
     - The ls -Z command displays the SELinux context of the file.
     - Example output: user_u:object_r:default_t:s0 confidential.txt
     - Change the context to allow access by specific processes: sudo chcon -t httpd_sys_content_t confidential.txt
   - What is happening?
     - Changes the file's security context so that only the httpd (web server) process can access it.

c. Test Policy Enforcement:
   - Try accessing the file through an unauthorized process.
   - Logs: sudo cat /var/log/audit/audit.log
   - What is happening?
     - SELinux denies access to unauthorized processes and logs the event in audit.log.

d. Results:
    - Access is granted or denied strictly based on SELinux policies, not ownership or file permissions.
    - Unauthorized processes will be denied access, even if they are run by the file owner.

3. Role-Based Access Control (RBAC)

- Create Roles (Groups):
    - Commands:
      sudo groupadd managers
      sudo usermod -aG managers alice
    - What is happening?
        - sudo groupadd managers: Creates a group called managers.
        - sudo usermod -aG managers alice: Adds Alice to the managers group.
- Set Permissions on a File:
    - Commands:
      touch manager_notes.txt
      sudo chown :managers manager_notes.txt
      sudo chmod 770 manager_notes.txt
    - What is happening?
        - touch manager_notes.txt: Creates a new file named manager_notes.txt.
        - sudo chown :managers manager_notes.txt: Changes the file's group ownership to managers.
        - sudo chmod 770 manager_notes.txt: Sets permissions:
            - Owner and group: Read, write, and execute.
            - Others: No permissions.
- Test Access:
    - As Alice (a member of the managers group):
      su alice
      cat manager_notes.txt
    - As another user (not in the managers group):
      su <other_user>
      cat manager_notes.txt
    - What is happening?
        - Alice can access the file because she belongs to the managers group.
        - Other users are denied access.
    - Results:
- Results:
    - Access to manager_notes.txt is controlled by group membership (role), not individual ownership.
    - This approach simplifies access control in organizations where users’ roles define their permissions.

Summary of Results

Model	Configuration	Result
DAC	File permissions (chmod, chown)	Only the file owner (Alice) can access the resource.
MAC	SELinux policies (chcon)	Access is strictly based on security policies, regardless of ownership.
RBAC	Group-based access (groupadd, chmod)	Access is based on user roles (group membership).

7. **Conclusion/Discussion:**
    - Discretionary Access Control (DAC) allows users to manage their own resources but can be vulnerable to insider threats.
    - Mandatory Access Control (MAC) enforces strict policies, ensuring high security but reducing flexibility.
    - Role-Based Access Control (RBAC) simplifies access management in hierarchical organizations.
    - Combining these models can enhance overall security by addressing different use cases and requirements.

8. **References:**

    - William Stallings, Cryptography and Network Security: Principles and Practice.
    - Linux Foundation Documentation on SELinux: https://selinuxproject.org
    - Ubuntu Documentation on Access Control: https://help.ubuntu.com
    - NIST Guidelines for Role-Based Access Control: https://csrc.nist.gov

Information Security Lab

Experiment No.: 3

Security Policy Simulation

Experiment No. 3

1. **Aim:** To design and simulate confidentiality and integrity policies in an enterprise environment, ensuring that sensitive data remains protected and unauthorized modifications are prevented.

2. **Objectives:**

   - Understand the principles of confidentiality and integrity policies in security models.
   - Implement security policies using Linux file permissions, AppArmor, and SELinux.
   - Simulate unauthorized access attempts and integrity violations.
   - Evaluate the effectiveness of implemented policies by monitoring security logs.
   - Analyze and compare the application of security policies in different real-world scenarios.

3. **Outcomes:**

   - Students will demonstrate knowledge of confidentiality and integrity policies.
   - Ability to implement file access control policies in an operating system.
   - Evaluate policy enforcement using log files and security auditing tools.
   - Gain hands-on experience with SELinux/AppArmor policies.
   - Understand the difference between DAC, MAC, and RBAC in security policy enforcement.

4. **Hardware / Software Requirements:**

   - **Hardware:**
     - A computer system with at least 4GB RAM, 20GB Disk space.
     - Virtualization enabled (for VM testing).
   - **Software:**
     - Ubuntu 22.04 LTS or later (installed on VMware or VirtualBox).
     - SELinux/AppArmor (Linux Mandatory Access Control).
     - Auditd (for monitoring security logs).
     - Text editors (nano or vim).
     - File system tools (chmod, chown, lsattr).

5. **Theory:**

   Security policies define rules and mechanisms that control access to data and system resources. They fall under two main categories:

   A. **Confidentiality Policies**

      - Aim to prevent **unauthorized access** to sensitive information.

- Example: **Bell-LaPadula Model** (No Read Up, No Write Down).
- Implementation in Linux:
  - **Discretionary Access Control (DAC)** → `chmod`, `chown`
  - **Mandatory Access Control (MAC)** → SELinux, AppArmor

B. **Integrity Policies**

- Ensure that data is not modified by unauthorized users.
- Example: Biba Model (No Write Up, No Read Down).
- Implementation in Linux:
  - Immutability Flags → chattr
  - Auditd Logging → Tracks unauthorized modifications

C. **Integrity Policies**

1. **Discretionary Access Control (DAC)**
   - Users set access control on files (e.g., chmod 600).
2. **Mandatory Access Control (MAC)**
   - System enforces access restrictions via SELinux/AppArmor.
3. **Role-Based Access Control (RBAC)**
   - Permissions are granted based on user roles.

6. Procedure/Program:

Step 1: Setup File Access Policies (Confidentiality)

1. Create a Confidential File:
   ```
   touch /home/student/secret.txt
   echo "Confidential Data: Do not share!" > /home/student/secret.txt
   ```

2. Apply Strict File Permissions (DAC)
   ```
   sudo chmod 600 “/home/student/secret.txt”
   sudo chown root:root “/home/student/secret.txt”
   ```

3. Test Unauthorized Access
   ```
   sudo -u nobody cat /home/student/secret.txt
   ```

   Expected Outcome: Permission denied

Step 2: Enforce Integrity Policies Using Immutability

1. Enable File Immutability (Biba Model)
   ```
   sudo chattr +i /home/student/secret.txt
   ```

2. Test Integrity Policy
   ```
   sudo echo "Unauthorized Change" >> /home/student/secret.txt
   ```

   Expected Outcome: Operation not permitted

3. Remove Immutability (Only When Required)
   sudo chattr -i /home/student/secret.txt

Step 3: Implement Mandatory Access Control (MAC) using SELinux/AppArmor

1. Check SELinux/AppArmor Status
   sestatus # For SELinux
   sudo aa-status # For AppArmor

   If disabled, enable SELinux:
   sudo setenforce 1

2. Apply SELinux Labeling (MAC)

   sudo semanage fcontext -a -t httpd_sys_content_t "/home/student/secret.txt"
   sudo restorecon -v /home/student/secret.txt

   Expected Outcome:
   - The file can only be accessed by HTTP services (if allowed).
   - Prevents unauthorized processes from reading/modifying it.

Step 4: Monitor Policy Violations Using Auditd

1. Install & Enable Auditd
   sudo apt install auditd -y
   sudo systemctl start auditd
   sudo systemctl enable auditd
2. Add Audit Rule for Unauthorized Access
   sudo auditctl -w /home/student/secret.txt -p war -k confidential_access

3. Check Audit Logs
   sudo ausearch -k confidential_access --start today

7. Conclusion/Discussion:

- Confidentiality policies were implemented using DAC (chmod) and MAC (SELinux/AppArmor).
- Integrity policies were enforced using immutable file attributes (chattr +i).
- Auditd logs captured policy violations for monitoring.
- This lab demonstrated how security models like Bell-LaPadula & Biba are applied in real-world operating systems.

8. References:

- NIST Special Publication 800-53 – Security and Privacy Controls for Information Systems.
- Bell-LaPadula Model – Confidentiality Policy (No Read Up, No Write Down).
- Biba Integrity Model – Data Integrity (No Write Up, No Read Down).
- SELinux Documentation: https://access.redhat.com/documentation/en-us/red_hat_enterprise_linux/7/html/selinux_users_and_administrators_guide/
- Ubuntu AppArmor Guide: https://wiki.ubuntu.com/AppArmor

Information Security Lab

Experiment No.: 4

Secure System Design

Experiment No. 4

1. **Aim:** To implement Identity-Based Access Control (IBAC) using Public Key Infrastructure (PKI), where users will authenticate using digital certificates, and simulate the confinement problem to ensure users can access only authorized resources.

2. **Objectives:**

   - Implement IBAC using PKI by configuring digital certificates for authentication.
   - Set up a Certificate Authority (CA) and issue user certificates.
   - Configure Apache Web Server to restrict access using certificates.
   - Create a protected directory requiring authentication via certificates.
   - Simulate the Confinement Problem using a Python script to ensure access restrictions based on clearance levels.

3. **Outcomes:**
   - Ability to set up and manage digital certificates using OpenSSL.
   - Secure a web server by enforcing certificate-based authentication.
   - Implement a simulated access control system using clearance levels.
   - Understand real-world application of IBAC and confinement problem in security.

4. **Hardware/Software Required:**
   - A computer or virtual machine (VM) with at least:
     - Processor: 1 GHz or faster
     - RAM: 4 GB
     - Disk Space: 20 GB free
   - **Software**:
     - Ubuntu 22.04 LTS (or later).
     - OpenSSL (for PKI setup and digital certificates)
     - Apache Web Server (for secure access control).
     - Python3 (for simulating the confinement problem).

5. **Theory:**

   What is Identity-Based Access Control (IBAC)?
   IBAC is a security model where users authenticate via digital certificates issued by a trusted Certificate Authority (CA). Only users with valid certificates can access protected resources.

   - **What is the Confinement Problem?**
     The confinement problem ensures that a user cannot access resources beyond their assigned clearance level. It applies the principle of least privilege, restricting unauthorized access.

   - **How PKI is Used for Access Control?**
     - Public Key Infrastructure (PKI) enables authentication and secure access through digital certificates.

- Apache Web Server is configured to accept only valid certificates for certain protected resources.

6. **Procedure/Program:**

Part 1: Identity-Based Access Using PKI

Step 1: Install Required Software

1. Update and Upgrade Ubuntu

   sudo apt update && sudo apt upgrade -y

2. Install OpenSSL for Certificate Management

   sudo apt install openssl -y
   openssl version # Verify installation

3. Install Apache Web Server

   sudo apt install apache2 -y
   sudo systemctl start apache2
   sudo systemctl enable apache2

   Test Apache: Open http://localhost in your browser. You should see the Apache default page.

Step 2: Generate Public/Private Key Pair

1. Generate the User's Private Key

   openssl genpkey -algorithm RSA -out user_private.key

2. Extract the Public Key

   openssl rsa -pubout -in user_private.key -out user_public.key

Step 3: Create & Sign a User Certificate

1. Create a Certificate Signing Request (CSR)

   openssl req -new -key user_private.key -out user.csr

   When prompted, enter details like:

   Country Name: US
   Organization Name: SecureLab
   Common Name: User1

2. Create a Self-Signed Certificate

   openssl x509 -req -in user.csr -signkey user_private.key -out user.crt

Step 4: Set Up a Certificate Authority (CA)

1. Create the CA Private Key

   openssl genpkey -algorithm RSA -out ca_private.key

2. Create the CA Certificate

   openssl req -key ca_private.key -new -x509 -out ca_certificate.crt

3. Sign the User's Certificate Using the CA

   openssl x509 -req -in user.csr -CA ca_certificate.crt -CAkey ca_private.key -CAcreateserial -out user_signed.crt

   Now, user_signed.crt is issued and signed by the CA.

Step 5: Configure Apache for Identity-Based Access

1. Enable SSL Module

   sudo a2enmod ssl

2. Configure SSL Certificate in Apache

   Edit the Apache SSL configuration:

   sudo nano /etc/apache2/sites-available/default-ssl.conf

   Replace <VirtualHost *:443> section with:

```
<VirtualHost *:443>
   DocumentRoot /var/www/html
   SSLEngine on
   SSLCertificateFile /etc/ssl/certs/user_signed.crt
   SSLCertificateKeyFile /etc/ssl/private/user_private.key
   SSLCACertificateFile /etc/ssl/certs/ca_certificate.crt
   SSLVerifyClient require
   SSLVerifyDepth 1
</VirtualHost>
```

3. Restart Apache

   sudo systemctl restart apache2

Step 6: Create a Protected Directory

1. Create a Secure Folder

   ```
   sudo mkdir /var/www/html/secret_data
   echo "This is a secret file." | sudo tee /var/www/html/secret_data/secret.txt
   ```

2. Restrict Access to the Directory

   Apache will require valid certificates to access /secret_data/.
   Apache is now configured for IBAC!

Step 7: Simulate the Confinement Problem

1. Create Python Script

   ```
   nano access_control.py
   ```

2. Add the Following Code

   ```
   # Function to check access based on clearance levels
   def check_access(user_clearance, resource_clearance):
   if user_clearance >= resource_clearance:
       return "Access Granted"
   else:
       return "Access Denied"

   # Clearance Levels
   clearance_levels = {
   1: "Top Secret",
   2: "Secret",
   3: "Confidential",
   4: "Unclassified"
   }

   # Get user input
   print("Available Clearance Levels:")
   for level, name in clearance_levels.items():
       print(f"{level}: {name}")

   user_clearance = int(input("Enter your clearance level (1-4): "))
   resource_clearance = int(input("Enter the clearance level of the resource (1-4):
         "))

   # Check Access
   access_result = check_access(user_clearance, resource_clearance)

   # Display Result
   print(f"Your Clearance Level: {clearance_levels[user_clearance]}")
   print(f"Resource Clearance Level: {clearance_levels[resource_clearance]}")
   ```

```
print(access_result)
```

3. Code Run the Script

```
python3 access_control.py
```

7. Conclusion/Discussion:

- Implemented Identity-Based Access Control (IBAC) using PKI.
- Configured Apache for certificate authentication.
- Created a protected directory with access restrictions.
- Simulated the confinement problem using a Python script.

Successfully designed a secure access system!

8. References:

- Public Key Infrastructure (PKI) Standards – NIST
- Apache SSL Documentation – https://httpd.apache.org/docs/
- Python Security Guide – https://docs.python.org/3/library/security.html

Information Security Lab

Experiment No.: 5

Vulnerability and Intrusion Detection

Experiment No. 5

1. **Aim:** The objective of this lab is to configure Intrusion Detection Systems (IDS) such as Snort and Suricata, and perform vulnerability analysis using tools like Nmap and Nikto to identify security weaknesses and detect potential attacks in an Ubuntu Virtual Machine (VM) environment.

2. **Objectives:**
   - To install and configure Snort and Suricata as Intrusion Detection Systems (IDS).
   - To perform vulnerability scanning using Nmap and Nikto.
   - To simulate intrusion attempts such as port scanning and brute-force attacks.
   - To analyze intrusion logs and implement security hardening techniques.

3. **Outcomes:**
   - Students will be able to deploy IDS tools to monitor and detect attacks.
   - Students will perform vulnerability assessment using scanning tools.
   - Students will analyze attack logs and implement defensive security measures.

4. **Hardware/Software Required:**
   - **Hardware:**
     - Ubuntu Virtual Machine (VM) with:
       - CPU: 2 GHz or higher
       - RAM: 4 GB or more
       - Disk Space: 20 GB free
   - **Software:**
     - Snort (IDS)
     - Nmap (Network Vulnerability Scanner)
     - Nikto (Web Application Scanner)
     - Hydra (Brute-force attack simulator)
     - Apache Web Server (for testing purposes)

5. **Theory:**

   A. **Intrusion Detection System (IDS)**

   An Intrusion Detection System (IDS) is a cybersecurity mechanism designed to monitor and analyze network or system activities for malicious behavior. It helps in detecting unauthorized access, policy violations, and potential cyber threats. When suspicious activity is detected, the IDS generates alerts to notify administrators.

   Types of IDS

   IDS can be classified into the following categories:

   1. Network-based Intrusion Detection System (NIDS)
      - Monitors network traffic in real time to detect suspicious patterns.
      - Examines packets moving across the network and identifies potential attacks.
      - Examples: Snort, Suricata.

2. Host-based Intrusion Detection System (HIDS)
   - Monitors activities on a specific host or system, including file integrity, system logs, and user activity.
   - Detects threats originating from within the system.
   - Examples: OSSEC, Tripwire.

Working of IDS
- Signature-Based Detection: Compares traffic against known attack patterns.
- Anomaly-Based Detection: Detects deviations from normal behavior using machine learning or statistical analysis.

An IDS helps in early threat detection and allows administrators to take proactive security measures.

B. Vulnerability Analysis

Vulnerability Analysis is the process of identifying security weaknesses in a network, system, or application. It helps security teams understand potential attack surfaces and take corrective actions.

Common Vulnerability Analysis Tools

1. Nmap (Network Mapper)
   - A powerful open-source tool used for network scanning.
   - Helps in identifying live hosts, open ports, and running services on a target machine.
   - Used for reconnaissance and penetration testing.
   - Example Usage: nmap -sV 192.168.1.1 (Scans for open ports and detects service versions on the target IP.)
2. Nikto
   - A web vulnerability scanner designed to detect security flaws in web applications.
   - Checks for outdated software versions, misconfigurations, and potential exploits.
   - Example Usage: nikto -h http://target.com (Scans the target website for vulnerabilities.)

By using these tools, administrators can proactively identify weaknesses and implement necessary security measures.

C. Attack Simulation

Attack simulation is an essential part of cybersecurity testing, allowing security professionals to understand how an attacker might exploit system vulnerabilities. Two common types of attack simulations are:

1. Port Scanning Attack
   - Definition: A technique used to identify open ports on a target machine.
   - Objective: Attackers use this method to map network services and find potential entry points.
   - Tool Used: Nmap
   - Example Command: nmap -p 1-65535 -T4 192.168.1.1 (Scans all ports on

the target IP to identify open services.)

2. Brute-Force Attack
   - Definition: A trial-and-error method used to guess passwords and gain unauthorized access to a system.
   - Objective: Attackers try multiple username-password combinations until they succeed.
   - Tool Used: Hydra
   - Example Command: hydra -L users.txt -P passwords.txt ssh://192.168.1.10 (Attempts SSH login on the target using a list of usernames and passwords.)

Brute-force attacks can be detected and prevented using:
- Intrusion Detection Systems (IDS) such as Snort.
- Account Lockout Policies that limit login attempts.
- Multi-Factor Authentication (MFA) for added security.

6. Procedure/Program:

Step 1: Update Ubuntu and Install Required Packages

1. Update the package repository:
   sudo apt update && sudo apt upgrade -y

2. Install security tools:
   sudo apt install snort suricata nmap nikto hydra apache2 -y

3. Enable Apache web server:
   sudo systemctl start apache2
   sudo systemctl enable apache2

Part 1: Configuring Intrusion Detection Systems (IDS)

Step 2: Configure Snort IDS

1. Verify Snort Installation:
   snort -V

2. Edit Snort Configuration File:
   sudo nano /etc/snort/snort.conf

   - Set the HOME_NET to match your network:
     ipvar HOME_NET 10.10.10.0/24
     preprocessor sfportscan: proto { all } scan_type { all } sense_level { low }
     output alert_fast: /var/log/snort/alert
3. Run Snort in IDS Mode:
   sudo snort -A console -c /etc/snort/snort.conf -i eth0

   (eth0: interface name, check ifconfig for the interface name and IP address)

Part 2: Performing Vulnerability Analysis

Step 3: Scan for Open Ports Using Nmap

1. Identify the Target IP Address:
   ip a

2. Run Nmap Scan:
   nmap -sV -p- 10.10.10.6 (Check for your system IP address here)

Step 4: Web Vulnerability Scan Using Nikto

1. Run Nikto Scan:
   nikto -h http://10.10.10.6 (Check for your system IP address here)

Part 3: Performing Vulnerability Analysis

Step 5: Simulate Port Scanning Attack

1. Run an aggressive Nmap scan:
   nmap -A 10.10.10.6 (Check for your system IP address here)

2. Check Snort Logs:
   sudo cat /var/log/snort/alert

Step 6: Simulate Port Scanning Attack

1. Run Hydra Brute-Force Attack on SSH:
   hydra -L users.txt -P passwords.txt ssh://10.10.10.6

   (Please create the user.txt and passwords.txt file by using mkdir command then Add some sample user as “test” and password as “toor” on the 10.10.10.6 system. Perform this attack from another VM like Kali Linux with ip as 10.10.10.4. Please remember all the IPs given may vary from system to system)

2. Check Snort Rules:
   sudo nano /etc/snort/snort.conf

   include $RULE_PATH/local.rules (Look for this line in the configuration file)

   sudo nano /etc/snort/rules/local.rules (Edit this file)

   alert tcp any any -> $HOME_NET any (msg:"Port scan detected"; flags:S; threshold: type threshold, track by_src, count 10, seconds 60; sid:1000001; rev:1;)

   alert tcp any any -> $HOME_NET 22 (msg:"SSH brute force attack detected"; flow:to_server,established; detection_filter:track by_src, count 5, seconds 30;

sid:1000002; rev:1;)

(Add these 2 rules in the local.rules file)

3. Check Snort Logs:
   sudo cat /var/log/snort/alert

Part 4: Intrusion Analysis and Prevention

Step 7: Analyze IDS Logs

1. View Snort Alerts:
   sudo tail -f /var/log/snort/alert

Step 8: Mitigate Vulnerabilities

1. Block Attacker's IP Address:
   sudo ufw deny from 10.10.10.4

2. Harden SSH Configuration:
   sudo nano /etc/ssh/sshd_config (Please change the following in the file)

   PermitRootLogin no
   AllowUsers test

3. Restart SSH:
   sudo systemctl restart ssh

7. Conclusion/Discussion:

- Snort IDS successfully detected simulated attacks.
- Nmap and Nikto identified system vulnerabilities.
- Attack logs provided useful insights for intrusion detection.
- Implementing firewall rules and SSH hardening helped prevent attacks.

8. References:

- Snort Official Documentation: https://www.snort.org/
- Nmap Security Scanner: https://nmap.org/
- Nikto Web Scanner: https://hackertarget.com/nikto-website-scanner/

Information Security Lab

Experiment No.: 6

Data Privacy and Encryption

Experiment No. 6

1. **Aim:** To implement data privacy and encryption techniques in a simulated environment to protect sensitive data using cryptographic methods, anonymization techniques, and access control mechanisms.

2. **Objectives:**

    - Understand the principles of data privacy, encryption, and anonymization.
    - Implement encryption using AES, RSA, and hashing for securing sensitive data.
    - Simulate data anonymization using data masking, tokenization, and generalization.
    - Evaluate the impact of encryption on data security and privacy.
    - Analyze real-world privacy threats and solutions in compliance with GDPR, HIPAA, and other data protection regulations.

3. **Outcomes:**

    - Learners will understand privacy risks and encryption techniques.
    - Ability to encrypt and decrypt data using AES & RSA.
    - Implement and analyze data anonymization techniques.
    - Simulate privacy-preserving methods for secure data sharing.
    - Ensure compliance with data protection policies.

4. **Hardware/Software Required:**
    - **Hardware:**
        - Computer with at least 4GB RAM, 20GB storage.
        - Virtualization enabled (for testing secure data storage).
    - **Software:**
        - Ubuntu 22.04 LTS (or Windows with Python)
        - Python 3+ (with Cryptography & Pandas libraries)
        - OpenSSL (for encryption)
        - MySQL/PostgreSQL (for database encryption)
        - Jupyter Notebook (for analyzing anonymized data)

5. **Theory:**

    A. **Data Privacy**
    Data privacy involves protecting sensitive information from unauthorized access while maintaining usability. Techniques include:

    - Encryption (AES, RSA)
    - Access Control
    - Data Anonymization
    - Data Masking

B. Encryption Techniques

Encryption converts readable data (plaintext) into an unreadable format (ciphertext) using an algorithm and a key.

1. Symmetric Encryption (AES - Advanced Encryption Standard)
   - Uses one key for encryption & decryption.
   - Faster, but key management is complex.

2. Asymmetric Encryption (RSA - Rivest-Shamir-Adleman)
   - Uses two keys: Public (encrypt) and Private (decrypt).
   - Slower, but more secure for data exchange.

C. Data Anonymization Techniques

Used to protect personal data while maintaining usability.

1. Data Masking
   - Replaces sensitive data with random characters.
   - Example: Credit Card: 1234-5678-9012-3456 → ****-****-****-3456.
2. Tokenization
   - Replaces sensitive data with a token (random string).
   - Example: user_email@example.com → TK-78392
3. Generalization
   - Reduces precision to remove uniqueness.
   - Example: Birth Year instead of Full Date of Birth.

6. Procedure/Program:

Step 1: Update Ubuntu and Install Required Packages

1. Update the package repository:
   sudo apt update && sudo apt upgrade -y

2. Install Python & Required Libraries
   sudo apt install python3 python3-pip -y
   pip3 install cryptography pandas faker

   python3 --version
   pip3 list | grep -E "cryptography|pandas|faker"

Step 2: AES Encryption & Decryption

AES (Advanced Encryption Standard) is a symmetric encryption algorithm.

1. Generate an AES Key : (Create nano aes_key.py)

```
from cryptography.fernet import Fernet

# Generate AES key
key = Fernet.generate_key()
```

```
# Save the key in a file
with open("aes_key.key", "wb") as key_file:
        key_file.write(key)

print(f"Generated AES Key: {key.decode()}")
```

2. Encrypt a Message (Create nano aes_encrypt.py)

```
from cryptography.fernet import Fernet

# Load AES key
key = open("aes_key.key", "rb").read()
cipher = Fernet(key)

# Message to encrypt
message = "Confidential Data: Do not share!"

# Encrypt the message
encrypted_message = cipher.encrypt(message.encode())

        print(f"Encrypted Message: {encrypted_message.decode()}")
```

3. Decrypt the Message (Create nano aes_decrypt.py)

```
# Decrypt the message
decrypted_message = cipher.decrypt(encrypted_message).decode()

        print(f"Decrypted Message: {decrypted_message}")
```

Step 3: RSA Encryption & Decryption

1. Generate RSA Key Pair (Create nano rsa_key.py)

```
from cryptography.hazmat.primitives.asymmetric import rsa
from cryptography.hazmat.primitives import serialization

# Generate private key
private_key = rsa.generate_private_key(public_exponent=65537,
key_size=2048)

# Save private key
with open("rsa_private.pem", "wb") as f:
f.write(private_key.private_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PrivateFormat.TraditionalOpenSSL,
        encryption_algorithm=serialization.NoEncryption()
))
```

```
# Generate public key
public_key = private_key.public_key()
with open("rsa_public.pem", "wb") as f:
f.write(public_key.public_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PublicFormat.SubjectPublicKeyInfo
))

        print("RSA Key Pair Generated Successfully!")
```

2. Encrypt Data Using RSA Public Key (Create nano rsa_encrypt.py)

```
from cryptography.hazmat.primitives.asymmetric import padding
from cryptography.hazmat.primitives import hashes

# Load the public key
public_key = serialization.load_pem_public_key(open("rsa_public.pem",
"rb").read())

message = b"Secure Data Transfer"

# Encrypt using the public key
encrypted = public_key.encrypt(
message,
padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
)
)

print(f"Encrypted Data: {encrypted}")
```

3. Decrypt Data Using RSA Private Key (Create nano rsa_decrypt.py)

```
# Load the private key
private_key = serialization.load_pem_private_key(open("rsa_private.pem",
"rb").read(), password=None)

# Decrypt the message
decrypted = private_key.decrypt(
encrypted,
padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
)
)

print(f"Decrypted Data: {decrypted.decode()}")
```

Step 4: Data Anonymization

1. Install Faker Library

   ```
   sudo apt install faker
   ```

2. Masking Sensitive Data (Anonymization) (Create nano mask.py)

   ```
   import pandas as pd

   data = {"SSN": ["123-45-6789", "987-65-4321", "555-44-3333"]}
   df = pd.DataFrame(data)

   # Masking SSN (showing only last 4 digits)
   df["Masked_SSN"] = df["SSN"].str.replace(r"\d{3}-\d{2}", "***-**-", regex=True)

   print(df)
   ```

2. Generate Fake Data for Privacy Protection

   ```
   from faker import Faker

   fake = Faker()

   for _ in range(5):
       print(fake.name(), "-", fake.email(), "-", fake.phone_number())
   ```

Step 5: Test and Verify Data Privacy

- Run encryption and anonymization scripts.
- Try to decrypt and view masked/anonymized data.
- Observe that data security policies prevent unauthorized access.

7. Conclusion/Discussion:

- Encryption (AES & RSA) ensures confidentiality of sensitive data.
- Data Anonymization (masking, tokenization, generalization) helps comply with privacy regulations.
- Simulated attacks confirmed data security measures are effective.

8. References:

- GDPR & HIPAA Compliance Guidelines
- AES & RSA Cryptography Standards - NIST
- Python Cryptography Docs: https://cryptography.io/

Information Security Lab

Experiment No.: 7

Digital Forensics Basics

Experiment No. 7

1. **Aim:** To simulate digital forensic techniques including evidence acquisition, analysis, and integrity verification using tools like dd, fls, and mount, performed within a Ubuntu/Kali VM on VMware Workstation.

2. **Objectives:**

    - Set up a forensic lab environment using VMware and Ubuntu/Kali.
    - Simulate evidence acquisition through disk imaging.
    - Perform file system analysis using forensic tools.
    - Understand and use hash functions to ensure evidence integrity.
    - Apply digital forensic principles in a controlled lab setting.

3. **Outcomes:**

    - Create a forensic disk image using dd.
    - Use Sleuth Kit tools (fls, mmls, etc.) for disk analysis.
    - Mount a forensic image in read-only mode.
    - Understand and maintain the chain of custody.
    - Apply hashing (MD5, SHA256) to verify image integrity.

4. **Hardware/Software Required:**

    - **Hardware:**
        - Computer with at least 4GB RAM, 20GB storage.
        - Virtualization enabled (for testing secure data storage).
    - **Software:**
        - Ubuntu 22.04 LTS / Kali Linux
        - Sleuthkit
        - Wireshark

5. **Theory:**

    Digital Forensics
    Digital Forensics refers to the process of collecting, analyzing, and preserving digital evidence in a legally acceptable manner.

    - Forensic Imaging
      Creating a bit-for-bit copy of a disk using tools like dd, ensuring the original evidence remains unaltered.

    - Filesystem Analysis
      Using tools like fls, mmls to examine deleted files, directory structures, and partition layouts.

    - Hashing
      Using md5sum or sha256sum to verify that no modifications were made to

the evidence during analysis.

6. **Procedure/Program:**

Step 1: Create a Ubuntu VM / Kali VM

Step 2: Add a Second Virtual Disk

- Power off the VM.
- Go to VM → Settings → Add → Hard Disk.
- Choose SCSI, 10 GB, and click Finish.
- Power on the VM.

Step 3: Identify the New Disk

```
lsblk
```

Expected Output:

sda (Ubuntu/Kali disk)
sdb (Attached disk for forensic image)

Step 4: Format and Populate the Secondary Disk

```
sudo mkfs.ext4 /dev/sdb
sudo mkdir /mnt/evidence
sudo mount /dev/sdb /mnt/evidence
echo "This is confidential evidence." | sudo tee /mnt/evidence/evidence.txt
sudo umount /mnt/evidence
```

Step 5: Create Forensic Image Using dd

```
sudo dd if=/dev/sdb of=forensic_image.img bs=4M status=progress
```

Step 6: Check Image Format

```
file forensic_image.img
```

Step 7: Verify Integrity with Hashing

```
md5sum forensic_image.img
sha256sum forensic_image.img
```

Step 8: Mount the Forensic Image (Read-Only)

```
sudo mkdir /mnt/forensic
sudo mount -o loop,ro forensic_image.img /mnt/forensic
ls /mnt/forensic
```

Step 9: Use fls for Deleted File Recovery

```
fls -f ext4 -r forensic_image.img
```

Step 10: Unmount After Analysis

```
sudo umount /mnt/forensic
```

7. Conclusion/Discussion:

This lab successfully demonstrated:

- Evidence acquisition using dd.
- Integrity assurance using hash functions.
- File recovery and analysis using Sleuth Kit tools.
- Basic understanding of disk structures and mounting forensic images.

8. References:

- NIST Guide to Integrating Forensic Techniques into Incident Response (SP 800-86)
- The Sleuth Kit & Autopsy Documentation
- 🔗 https://www.sleuthkit.org
- Ubuntu File System Forensics Guide
- Carrier, Brian. “File System Forensic Analysis” (Addison-Wesley)

Information Security Lab

Experiment No.: 8

Network and Program Security

Experiment No. 8

1. **Aim:** To understand and implement network communication analysis and program security techniques by capturing and securing traffic, analyzing vulnerabilities, and applying countermeasures in a virtual lab setup.

2. **Objectives:**

   - Set up a secure networked environment using VMs.
   - Analyze network packets using tools like Wireshark and tcpdump.
   - Identify potential vulnerabilities in network protocols.
   - Use firewall rules (UFW) to restrict communication.
   - Understand basic buffer overflow and secure coding practices.
   - Apply encryption (SSL/TLS) and secure shell (SSH) for safe communication.

3. **Outcomes:**

   After completing the lab, students will be able to:

   - Monitor and analyze real-time network communication.
   - Configure basic firewalls to secure systems.
   - Understand common attacks on networks and programs.
   - Write and test secure code using safe programming practices.
   - Encrypt communications using SSH, SSL/TLS.

4. **Hardware/Software Required:**

   - **Hardware:**
     - Computer with at least 4GB RAM, 20GB storage.
     - At least 2 VMs recommended for testing communications
   - **Software:**
     - Ubuntu 22.04 LTS / Kali Linux
     - Sleuthkit
     - Wireshark

5. **Theory:**

   - **Network Security**

     Involves protecting data in transit from interception, modification, or disruption using protocols like **SSH, HTTPS**, and firewalls.

   - **Program Security**

     Protects code from vulnerabilities like **buffer overflows**, **format string bugs**, or **command injections**. Secure coding practices are vital to reduce attack surfaces.

- **Traffic Monitoring**

Tools like Wireshark and tcpdump are used to **analyze packets**, detect anomalies, and understand the flow of information.

6. **Procedure/Program:**

Step 1: Create 2 VMs in VMware

- Ubuntu/Kali Linux (4 GB RAM each)
- Enable bridged or NAT networking to allow communication

Step 2: Start SSH Server (on one VM)

```
sudo apt install openssh-server -y
sudo systemctl enable ssh
sudo systemctl start ssh
```

Check IP:

```
ip a
```

Step 3: Connect via SSH (from other VM)

```
ssh username@<target_ip>
```

Note: Here the username & target ip is the VM name and its ip address. (Check the IP using ifconfig)

Accept the key and enter password. This simulates secure communication.

Step 4: Monitor Network using tcpdump

```
sudo tcpdump -i any port 22
```

Now try the SSH connection again and observe the output.

Step 5: Use Wireshark for Packet Capture

```
sudo wireshark &
```

Capture packets on eth0 or ens33

Apply filter: `tcp.port == 22`

Step 6: Scan Ports with nmap

```
nmap -sS -p 1-1000 <target_ip>
```

Observe open ports and services

Step 7: Secure Ports using UFW Firewall

```
sudo ufw enable
sudo ufw default deny
sudo ufw allow 22/tcp
sudo ufw allow 80/tcp
sudo ufw status verbose
```

Step 8: Buffer Overflow Simulation (C Program)

```
#include <stdio.h>
#include <string.h>

void vulnerable_function() {
    char buffer[10];
    printf("Enter input: ");
    gets(buffer);  // Unsafe
    printf("You entered: %s\n", buffer);
}

int main() {
    vulnerable_function();
    return 0;
    }
```

Save this as vuln.c (use command nano vuln.c)

Compile and Run:

```
gcc vuln.c -o vuln -fno-stack-protector -z execstack
./vuln
```

Fix:
Replace gets(buffer) with fgets(buffer, sizeof(buffer), stdin);

Step 9: Simulate HTTPS Request

curl https://www.google.co.in

curl -v https://www.google.co.in

Observe headers, encryption (SSL/TLS)

7. Conclusion/Discussion:

This lab walked through:

- Capturing and analyzing network traffic.
- Simulating secure communication using SSH.
- Detecting vulnerabilities using port scanners.
- Hardening the system using firewall and secure coding.

Students learned the basics of network and program-level threats and how to mitigate them.

8. References:

- Wireshark Documentation: https://www.wireshark.org
- OWASP Secure Coding Practices: https://owasp.org/www-project-secure-coding-practices
- Nmap Official Guide: https://nmap.org/book/
- Linux UFW Firewall: https://help.ubuntu.com/community/UFW
- Buffer Overflow Examples: https://exploit.education

Information Security Lab

Experiment No.: 9

Operating Systems Security

Experiment No. 9

1. **Aim:** To analyze and enhance the security posture of Linux (Ubuntu/Kali) and Windows operating systems by identifying vulnerabilities, configuring security features, managing user access, and applying system hardening techniques.

2. **Objectives:**

    - Understand OS-level security components and configurations.
    - Identify open ports, running services, and potential vulnerabilities.
    - Apply user and permission management.
    - Implement audit and logging mechanisms.
    - Use security tools like chkrootkit, Lynis, and auditd on Linux.
    - Enable and configure Windows Security, Defender, and Event Viewer (if using Windows).

3. **Outcomes:**
   After completing the lab, students will be able to:

    - Analyze vulnerabilities in an operating system.
    - Apply OS-level security configurations.
    - Configure user permissions and secure file access.
    - Audit system activity and investigate logs.
    - Use Linux and Windows tools to harden systems.

4. **Hardware/Software Required:**

    - **Hardware:**
        - Computer with at least 4GB RAM, 20GB storage.
        - At least 2 VMs recommended for testing communications
    - **Software:**
        - Ubuntu 22.04 LTS / Kali Linux

5. **Theory:**

   Operating System Security

   OS Security ensures confidentiality, integrity, and availability of resources and services on a system. It involves managing user access, securing files, updating software, and monitoring activities.

   ◇ Key Security Concepts

    - Principle of Least Privilege
    - Patch Management
    - Service Minimization
    - Firewall Configuration
    - Audit and Monitoring
    - Intrusion Detection

6. **Procedure/Program:**

Step 1: Check Running Services & Open Ports

```
sudo ss -tuln
sudo netstat -tulnp
```

Step 2: Use UFW (Firewall)

```
sudo ufw enable
sudo ufw default deny incoming
sudo ufw allow 22/tcp
sudo ufw status verbose
```

Step 3: Secure User Accounts

- Check users:

  cut -d: -f1 /etc/passwd

- Disable unused accounts:

  sudo usermod -L username (Here you can delete Alice)

- Set strong passwords:

  sudo passwd username (Use the password of Alice)

Step 4: File and Directory Permissions

- View permissions:

  ls -l /etc/shadow

- Modify permissions:

  sudo chmod 640 /etc/shadow

  sudo chown root:shadow /etc/shadow

Step 5: Rootkit Detection

sudo chkrootkit

sudo rkhunter --check

Step 6: Perform Security Audit with Lynis

sudo lynis audit system

Review suggestions at the end of the audit.

Step 7: Configure System Logging and Auditing

```
sudo auditctl -e 1

sudo systemctl enable auditd
```

To monitor file:

```
sudo auditctl -w /etc/passwd -p war -k passwd_changes
```

To view logs:

```
sudo ausearch -k passwd_changes
```

Step 8: Install and Scan with ClamAV Antivirus

```
sudo freshclam

sudo clamscan -r /home
```

7. Conclusion/Discussion:

This lab provides a practical approach to operating system hardening and monitoring using built-in and third-party tools. By completing this lab, students gain essential skills in OS-level threat analysis, detection, and mitigation across both Linux and Windows platforms.

8. References:

- Ubuntu Security Guide
- Lynis Security Tool
- Auditd Documentation
- chkrootkit Tool
- Microsoft Windows Security
- ClamAV Documentation

Information Security Lab

Experiment No.: 10

Database Security and Auditing

Experiment No. 10

1. **Aim:** To implement security controls and auditing mechanisms in a database system to protect data integrity, confidentiality, and availability.

2. **Objectives:**

    - Install and configure MySQL/MariaDB securely.
    - Apply user authentication and access control.
    - Enable auditing mechanisms to track activities.
    - Encrypt data at rest and in transit.
    - Understand and implement SQL injection protection.

3. **Outcomes:**
   After completing the lab, students will be able to:

    - Configure secure database users with role-based privileges.
    - Audit and log user activities in a database.
    - Identify potential security loopholes in database systems.
    - Implement best practices for database security.

4. **Hardware/Software Required:**

    - **Hardware:**
        - Computer with at least 4GB RAM, 20GB storage.
    - **Software:**
        - Ubuntu 22.04 LTS / Kali Linux
        - MySQL

5. **Theory:**

   Database security is the practice of protecting data against compromise, corruption, and loss. Key components include:

    - Authentication: Ensuring only authorized users access the database.
    - Authorization: Granting specific permissions to users.
    - Auditing: Logging user activities for accountability.
    - Encryption: Protecting sensitive data both at rest and in transit.
    - Vulnerability Management: Regularly updating and patching databases.

6. **Procedure/Program:**

Step 1: Install MySQL Server

```
sudo apt update
sudo apt install mysql-server -y
```

Step 2: Secure MySQL Installation

```
sudo mysql_secure_installation
```

- Set root password
- Remove anonymous users
- Disallow remote root login
- Remove test DB
- Reload privilege tables

Step 3: Login and Create a Secure User

```
sudo mysql -u root -p

CREATE USER 'secuser'@'localhost' IDENTIFIED BY 'StrongPassword!';
GRANT SELECT, INSERT ON testdb.* TO 'secuser'@'localhost';
FLUSH PRIVILEGES;
```

Step 4: Enable Auditing in MySQL

For MariaDB or MySQL 5.7+, use:

```
sudo apt install libmysqlclient-dev
```

Enable logging:

```
sudo nano /etc/mysql/mysql.conf.d/mysqld.cnf
```

Add:

```
general_log = 1
general_log_file = /var/log/mysql/mysql.log
```

Restart MySQL:

```
sudo systemctl restart mysql
```

Check log:

```
sudo tail -f /var/log/mysql/mysql.log
```

Step 5: Test SQL Injection Prevention

Create a Database and Table

Login to MySQL: mysql -u root -p

In the MySQL shell:

```
CREATE DATABASE testdb;
USE testdb;

CREATE TABLE users (
 id INT AUTO_INCREMENT PRIMARY KEY,
username VARCHAR(50),
 password VARCHAR(50)
);

INSERT INTO users (username, password) VALUES ('admin', 'admin123'),
('user1', 'pass123');

EXIT;
```

Create Flask App Files

Create a project folder and move into it:

```
mkdir sql_injection_demo
cd sql_injection_demo
```

Create a file: `app.py`

Add Python Code

```
from flask import Flask, request
import mysql.connector

app = Flask(__name__)

# Connect to database
def get_connection():
    return mysql.connector.connect(
        host="localhost",
        user="root",
        password="your_mysql_password",
        database="testdb"
    )

# Vulnerable route
@app.route('/login-vulnerable', methods=['GET', 'POST'])
def login_vulnerable():
    if request.method == 'POST':
        username = request.form['username']
```

```
        query = f"SELECT * FROM users WHERE username = '{username}'"
        conn = get_connection()
        cursor = conn.cursor()
        cursor.execute(query)
        result = cursor.fetchall()
        conn.close()
        return str(result) if result else "No user found"
    return '''
        <form method="post">
            Username: <input name="username"><br>
            <input type="submit">
        </form>
    '''

# Safe route using parameterized query
@app.route('/login-safe', methods=['GET', 'POST'])
def login_safe():
    if request.method == 'POST':
        username = request.form['username']
        conn = get_connection()
        cursor = conn.cursor()
        query = "SELECT * FROM users WHERE username = %s"
        cursor.execute(query, (username,))
        result = cursor.fetchall()
        conn.close()
        return str(result) if result else "No user found"
    return '''
        <form method="post">
            Username: <input name="username"><br>
            <input type="submit">
        </form>
    '''

if __name__ == '__main__':
    app.run(debug=True)
```

Replace `"your_mysql_password"` with your actual MySQL root password.

Run the Flask App → python3 app.py

Open browser and go to:

- http://localhost:5000/login-vulnerable
- http://localhost:5000/login-safe

Test SQL Injection
Try entering this in the vulnerable form: ' OR '1'='1

It should return all users — showing it's vulnerable.

Try the same in the safe version, and it should block the injection.

7. Conclusion/Discussion:

This lab demonstrated essential techniques to secure database systems. By applying access control, auditing, encryption, and best practices, students gained hands-on experience in protecting critical data assets from misuse or breach.

8. References:

- MySQL Secure Installation
- MariaDB Audit Plugin
- OWASP SQL Injection Guide
- Ubuntu MySQL Docs